

EPIC: Educate, Plan, Invest and Cultivate



1. Introdução

A aplicação **EPIC** visa integrar **gestão financeira** e **organização pessoal** numa única plataforma. Reunindo funcionalidades como controlo de receitas/despesas, agendamento de eventos críticos e recomendações inteligentes via IA, sendo o objetivo **automatizar tarefas repetitivas** e **fornecer insights úteis** para melhorar a saúde financeira do utilizador.

Nos dias atuais, muitas pessoas têm dificuldade em manter consistência no controlo das suas finanças e esquecem-se frequentemente de compromissos ou prazos importantes. A **EPIC** procura resolver esse problema com automatização e inteligência contextual.

Objetivos do Projeto

- Permitir o registo e categorização automática de transações financeiras.
 - Notificar utilizadores sobre eventos importantes (ex: pagamento de faturas).
 - Oferecer comparações históricas de gastos (meses/anos).
 - Automatizar a inserção de dados via OCR a partir de faturas.
 - Gerar relatórios mensais/anuais em PDF com análises personalizadas.
 - Fornecer recomendações de otimização de gastos via IA.
-

2. Apresentação do Negócio

Num contexto de crescente preocupação com a literacia financeira e organização pessoal, esta aplicação visa preencher a lacuna entre ferramentas demasiado simples (como folhas de cálculo) e plataformas demasiado técnicas (como softwares de contabilidade).

Âmbito e Enquadramento

- **Público-alvo:**

- Indivíduos e famílias que desejam controlar finanças e organizar compromissos críticos.

- **Diferenciais:**

- Integração de IA para insights personalizados.
- OCR para automatização de registos.
- Eventos "inteligentes" que sugerem lembretes com base em transações futuras.

- **Modelo de Negócio** (A repensar):

- **Freemium:** versão gratuita com funcionalidades essenciais + versão premium com funcionalidades avançadas como dashboards interativos, previsões baseadas em IA, exportações avançadas, relatórios personalizados e alertas inteligentes.
-

3. Levantamento de Requisitos

- **Funcionais:**

- Gestão de utilizadores (registar, autenticar via JWT).
- Registar receitas/despesas com categorias personalizáveis.
- Agendar eventos com notificações personalizadas (ex: 3 dias antes).
- Upload de faturas com extração automática de dados (OCR).
- Dashboard com gráficos de evolução financeira e outliers.
- Exportação de relatórios em PDF com sugestões da IA.
- Permitir editar e eliminar transações.
- Marcar transações como recorrentes.
- Enviar avisos caso tenha selecionado budget mensal/anual máximo.

- **Não Funcionais:**

- **Portabilidade:** Compatível com Android e iOS.
- **Performance:** Rápido tempo de resposta entre as requisições.
- **Segurança:** Criptografia de dados sensíveis e autenticação JWT.

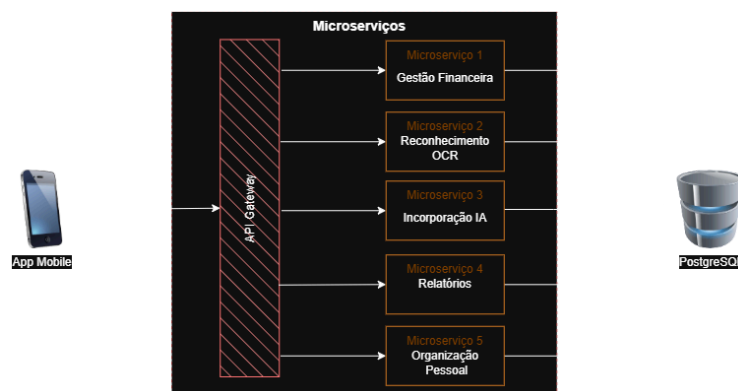
- **Usabilidade:** Interface intuitiva.

4. Stack Tecnológica

Backend	Spring Boot, Hibernate, PostgreSQL, JWT
Frontend	Flutter/React
OCR	Tesseract.js
IA	OpenAI GPT-4
Relatórios	Flying Saucer (PDF)
Testes	JUnit (Backend), Jest (Frontend), Swagger
Versionamento	Git, Github
CI/CD	Github Actions

4. Arquitetura, Design e Modelação

A aplicação adota uma arquitetura baseada em **microserviços** com um **API Gateway** central, proporcionando escalabilidade, manutenção isolada por domínio funcional e facilidade na integração de novas funcionalidades.



Componentes Principais

1. App Mobile

- Interface principal de interação do utilizador.
- Responsável por enviar e receber dados do backend através do API Gateway.

- Desenvolvido com foco na usabilidade, mobilidade e integração com notificações e uploads de documentos.

2. API Gateway

- Camada central de entrada para todas as comunicações entre o frontend e os microserviços.
- Responsabilidades:
 - Roteamento de requisições para os microserviços corretos.
 - Autenticação e autorização.
 - Gestão de taxa de requisições (rate limiting).
 - Centralização de logs e monitorização.

3. Microserviços

Cada microserviço é responsável por uma funcionalidade específica do sistema:

• **Microserviço 1 – Gestão Financeira**

- Gestão de despesas, receitas, categorias e contas.
- Cálculos de saldo e planeamento financeiro.

• **Microserviço 2 – Reconhecimento OCR**

- Processamento de faturas e recibos submetidos pelo utilizador.
- Conversão de imagem para texto utilizando OCR (Tesseract.js, por exemplo).

• **Microserviço 3 – Incorporação de IA**

- Geração de insights personalizados.
- Sugestões automáticas de poupança e análise de padrões de despesa.
- Potencial integração com modelos da OpenAI.

• **Microserviço 4 – Relatórios**

- Criação de dashboards e relatórios interativos.
- Detecção de outliers, gráficos mensais e comparativos.

• **Microserviço 5 – Organização Pessoal**

- Planeamento de tarefas e eventos com notificações.
- Integração com calendário e lembretes personalizados.

4. Base de Dados – PostgreSQL

- Base de dados relacional central utilizada por todos os microserviços.
- Cada microserviço pode ter o seu próprio esquema (schema) para garantir independência lógica e segurança de dados.

5. Estrutura do projeto

Aplicação baseada em microserviços, composta por múltiplos serviços independentes que colaboram entre si. Cada microserviço é responsável por um domínio específico da aplicação e comunica com os demais serviços através de APIs REST.

EPIC-API/

```
| — api-gateway/           # Ponto de entrada (Spring Boot)
| | — config/              # Configurações globais do gateway
| | — filters/             # Filtros e interceptors HTTP
| | — utils/               # Funções utilitárias
| | — uploads/             # Ficheiros recebidos temporários
| | — Application.java      # Ponto de entrada Spring Boot
|
| — microservices/
| | — finance-service/     # Gestão financeira
| | | — src/main/java/com/epic/finance/
| | | | — controller/      # Controladores REST
| | | | — service/         # Interfaces da lógica de negócio
| | | | — repository/      # Repositórios JPA
| | | | — entity/          # Entidades JPA
| | | | — dto/             # DTOs
| | | | — mapper/          # Conversores model ↔ dto
| | | | — config/          # Configurações específicas
| | | | — exception/       # Gestão de exceções
| | | — pom.xml
| |
| | — ai-service/          # Módulo IA
| | — planner-service/     # Planeamento e organização
| | — report-service/      # Gerar relatórios
| | — auth-service/        # Autenticações
|
| | — ocr-service/         # OCR em Node.js
| | | — src/
| | | | — controllers/     # Entrada API
| | | | — services/        # Lógica OCR
| | | | — routes/          # Endpoints
| | | | — utils/           # Utilitários
```

```

|   |   | uploads/      # Uploads temporários
|   |   |   index.ts    # Ponto de entrada Node.js
|   |   |   package.json
|
|   | shared/           # Código compartilhado
|   |   | src/main/java/com/epic/shared/
|   |   |   pom.xml
|
|   | docs/             # Documentação do projeto
|   |   | infrastructure/ # Scripts, configs para deploy e infra
|   |   |   .gitignore
|   |   |   README.md

```

Explicação:

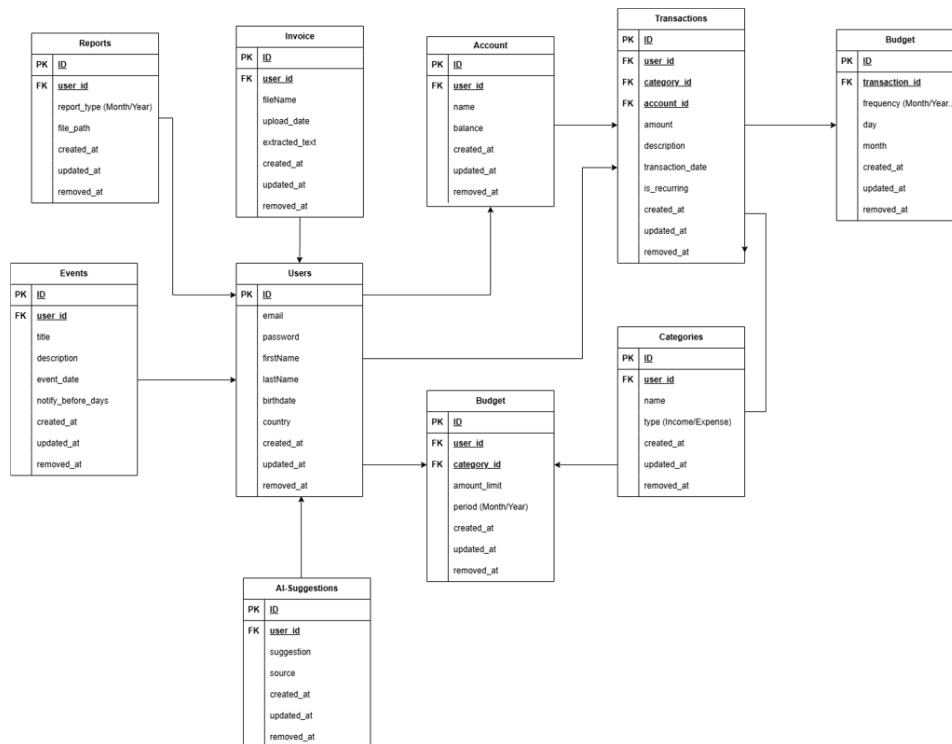
- **controller/**: Expõe endpoints REST. Não deve ter lógica de negócio nem comunicar diretamente com a base de dados.
- **service/**: Define as interfaces a utilizar na lógica da aplicação.
- **repository/**: Interfaces para acesso a dados, utiliza Spring Data JPA (ou outro ORM).
- **dto/ e mapper/**: Evitam expor entidades diretamente ao exterior. Boa prática para manter separação de preocupações.
- **exception/**: Ajuda a manter o código limpo e previsível em caso de erros.

Vantagens da Arquitetura:

- **Escalabilidade**: Cada microserviço pode crescer de forma independente.
 - **Manutenção facilitada**: Cada domínio tem o seu repositório lógico.
 - **Isolamento de falhas**: Erro num microserviço não derruba os restantes.
 - **Autonomia técnica**: Serviços podem evoluir com tecnologias diferentes se necessário (ex: Node.js no OCR).
-

6. Base de dados

Modelo Relacional:



7. Segurança

- **JWT Token**: Utilizado para autenticação de todas as requisições.
- **Acesso controlado**: Apenas o utilizador autenticado pode aceder aos seus próprios dados.
- **Sem roles**: A app não utiliza **RBAC**(Role-Based Access Control), visto que não há hierarquia de utilizadores.
- **Criptografia**: Dados sensíveis (ex: palavras-passe) serão encriptados com BCrypt ou equivalente.

8. Deploy e CI/CD

- **GitHub Actions**:
 - Pipeline CI(Continuous Integration) para testes e build dos microserviços.
 - Pipeline CD(Continuous Delivery) para deploy automatizado (em produção/future staging).
 - Deploy manual via GitHub environments(Master, Develop)

9. Monitorização e Logs

Fase Inicial

- Logs locais (Spring Boot, consola), através do Spring Boot Actuator para métricas básicas.

Fase Avançada (Futuro)

- **Logs Centralizados:** ELK Stack (Elasticsearch, Logstash, Kibana)
 - **Métricas e Monitorização:** Grafana
-

10. Estratégia de Testes

- **Testes Unitários:**
 - Backend: JUnit + Mockito
 - OCR: Jest
 - **Testes de Integração** (posteriormente):
 - Testcontainers (para PostgreSQL)
 - Spring Boot Test
 - **Testes de API:**
 - Swagger UI
-

11. Roadmap Futuro

Fase	Objetivos Principais
MVP	Login, registo, registo manual de despesas/receitas, visualização simples de dados
Fase 2	Upload de faturas (OCR), dashboard inicial
Fase 3	Geração de relatórios PDF, notificações personalizadas, IA com sugestões financeiras
Fase 4	Calendário inteligente, integração com eventos recorrentes (rendas, faturas, etc.)
Fase 5	Observabilidade, escalabilidade via Docker/Kubernetes, RBAC completo

Fase 6+	Funcionalidades premium, deploy em cloud pública, App Store/Play Store release
---------	--